

Estándar Protocolo CTF

CC8 2026

1. CONEXIÓN

1.1 Transporte

Define la tecnología encargada de mover los bytes entre las máquinas participantes y las garantías de entrega que esta ofrece. Todos los proyectos de la clase deben acordar el mismo transporte para que sus conexiones sean compatibles entre sí.

Se establece un esquema híbrido:

- **TCP** para toda la comunicación de la partida (unirse, moverse, tomar la bandera, estado del juego, fin de partida). Garantiza que los mensajes lleguen completos y en el orden en que se enviaron.
- **UDP** exclusivamente para el descubrimiento de servidores en la red local (desarrollado en la sección 1.3).

Requisitos para cada proyecto:

- Usar los sockets básicos que ya trae el lenguaje, sin librerías externas de conexión (por ejemplo: `socket` en Python, `net` en Go, `TcpClient` en C#, `net` en Node —nunca `ws`—, POSIX/Winsock en C++, `StreamPeerTCP` en Godot, `dart:io` en Dart).
- En **modo cliente**, abrir una única conexión TCP hacia el servidor y utilizarla para todo el intercambio de mensajes del juego.
- En **modo servidor**, aceptar y mantener conexiones TCP simultáneas de hasta 100 clientes.
- Mantener, además, un socket UDP independiente dedicado solo al descubrimiento.

1.2 Puertos

Establece los números de puerto en los que los servidores escuchan, tanto para el descubrimiento como para la partida. Separar ambos puertos evita que la búsqueda de servidores interfiera con el tráfico del juego en curso.

Se definen dos tipos de puerto:

- **Puerto de descubrimiento (UDP):** fijo e igual para los 16 proyectos de la clase — **8888**. Todo servidor debe escuchar ahí sin excepción.
- **Puerto de juego (TCP):** cada servidor lo elige libremente (por ejemplo, 8889) y no necesita coincidir entre proyectos.

Requisitos para cada proyecto:

- El servidor debe **anunciar su puerto TCP** dentro del mensaje de respuesta al descubrimiento, ya que el cliente no lo conoce de antemano.
- El cliente primero pregunta por el puerto UDP fijo (8888), lee el puerto TCP que el servidor le indica, y **con ese puerto** abre la conexión de juego.
- Ningún proyecto debe hardcodear un puerto TCP ajeno: siempre se obtiene dinámicamente de la respuesta de descubrimiento

1.3 Descubrimiento de servidores

Broadcast explanation

The client does not know the server IP, so it sends a special "Who is there?" request packet to the local network's broadcast address (usually 255.255.255.255 or your subnet's broadcast, like 192.168.1.255).

This packet is sent to every single device connected to that local network. The Minecraft server receives this request. It looks at the source IP of the incoming request (which is the client's IP) so it knows where to send the reply. The server sends a reply packet directly back to the client's IP.

When the client receives this reply, it checks the source IP of the incoming packet, which is the server's actual LAN IP. It then displays that server in the list.

Again, the client discovers the server's IP directly from the source address of the reply packet.

Descubrimiento de servidores

Describe el procedimiento mediante el cual un cliente localiza los servidores disponibles en la red local sin conocer sus direcciones IP de antemano. Este mecanismo depende del puerto UDP definido en la sección 1.2.

El descubrimiento sigue dos vías:

- **Automática (broadcast UDP):** el cliente envía una pregunta a toda la red y cada servidor activo responde de forma individual.
- **Manual (respaldo):** el cliente permite escribir directamente la IP de un servidor, para los casos en que el broadcast no funcione (router o WiFi que lo bloquee).

Requisitos para cada proyecto:

- El cliente debe enviar por broadcast (255.255.255.255:8888) un mensaje del tipo: `{"type":"discover","v":1}`.
- El servidor debe responder directamente al remitente con: `{"type":"server_info","v":1,"name":"...","tcp_port":8889,"state":"lobby","players":3}`.
- El cliente debe mostrar la lista de servidores encontrados y permitir al usuario elegir uno.
- El cliente debe incluir, sin excepción, la opción de conexión manual por IP como respaldo al broadcast

2. EL IDIOMA DE LOS MENSAJES

2.1 Delimitación de mensajes (framing)

Especifica la regla que permite identificar dónde termina un mensaje y dónde comienza el siguiente. Este punto es indispensable porque TCP entrega los datos como un chorro continuo de bytes, sin ninguna separación natural entre mensajes — dos mensajes pueden llegar pegados en una sola lectura, o uno solo puede llegar partido en dos lecturas distintas.

Se define la siguiente regla:

- **Un mensaje = una línea.** Cada mensaje es un texto JSON completo, seguido del carácter de salto de línea (`\n`).
- El JSON de un mensaje **no puede contener saltos de línea internos** — debe escribirse siempre "aplastado" en una sola línea (sin indentación ni formato bonito).
- El salto de línea (`\n`) es exclusivamente el separador entre mensajes; nunca debe aparecer dentro del contenido de un campo.

Procedimiento obligatorio para leer mensajes:

1. Ir acumulando los bytes que llegan por el socket en un buffer (una especie de "cajón" temporal).
2. Cada vez que llegue un `\n` dentro de ese buffer, cortar ahí: todo lo acumulado ANTES del `\n` es un mensaje completo.
3. Convertir ese texto cortado en un objeto JSON (parsear).
4. Lo que quede DESPUÉS del `\n` se conserva en el buffer, porque puede ser el inicio del siguiente mensaje (o estar incompleto todavía).
5. Repetir mientras la conexión siga abierta.

Aclaración importante: esta regla aplica únicamente a mensajes enviados por TCP. Los mensajes de descubrimiento, enviados por UDP, se transmiten como paquetes completos en un solo paso y no requieren el carácter `\n` ni el proceso de buffer.

2.2 Formato y codificación

Define el lenguaje en que se escriben los mensajes y la codificación de caracteres utilizada, de modo que todos los proyectos interpreten la misma información de forma idéntica, sin importar el lenguaje de programación en que estén escritos.

Se establecen las siguientes reglas obligatorias:

- **Formato: JSON.** Todo mensaje del protocolo debe ser un objeto JSON válido, no un texto libre ni un formato inventado por cada proyecto.
- **Codificación: UTF-8.** Todo el texto (nombres de jugadores, motivos de error, etc.) debe codificarse y decodificarse en UTF-8 en ambos extremos de la conexión.



campo identificador obligatorio

```
{ "type": "join", ... }
```

todo mensaje lleva `"type"` como texto — ninguno puede omitirlo

Nº

valores numéricos

```
"x": -1
```

```
"x": "-1"
```

posición y dirección (`x` , `y` , `dir`) son números, nunca texto



estructura plana

```
{ "welcome", "config": { "map_size": 1000 } }
```

```
{ "config": { "map": { "size": 1000 } } }
```

máximo 2 niveles de profundidad — mantiene el parseo simple en todos los lenguajes

Campo identificador obligatorio. Todo mensaje, sin excepción, debe incluir un campo llamado `"type"` como texto (string), cuyo valor identifica de qué mensaje se trata. Ningún mensaje puede omitir este campo.

Valores numéricos. Los campos de posición y dirección (`x`, `y`, `dir`) se representan como números, nunca como texto.

Estructura plana. Los mensajes no deben anidar más de dos niveles de profundidad (por ejemplo, `config` dentro de `welcome` es aceptable; un tercer nivel dentro de `config` no lo es), para mantener el parseo simple en todos los lenguajes.

2.3 Catálogo de mensajes

Aquí definimos la lista completa y cerrada de todos los mensajes que va a manejar el protocolo: cuáles existen, qué campos lleva cada uno, en qué dirección viajan (cliente→servidor o servidor→cliente) y en qué momento de la partida se envían.

Catálogo general de mensajes

Protocolo de Comunicación · Captura la Bandera · vista resumida por concepto

tipo	dirección	fase	propósito	campos principales
discover	C → UDP	cualquiera	buscar servidores en la red	v
server_info	S → UDP	cualquiera	anunciar el servidor encontrado	v, name, tcp_port, state, players
join	C → S	lobby	unirse a la partida	v, name
input	C → S	playing	comunicar hacia dónde se mueve	dir
interact	C → S	playing	tomar o robar la bandera	(ninguno)
welcome	S → C	lobby	asignar identidad y constantes	player_id, config
lobby	S → C	lobby	lista de jugadores en espera	players
countdown	S → C	countdown	mostrar la cuenta regresiva	seconds
start	S → C	countdown→playing	iniciar la partida	(ninguno)
state	S → C	playing	replicar el mundo del juego	flag, players
game_over	S → C	playing→finished	anunciar al ganador	winner
error	S → C	cualquiera	rechazar una acción inválida	reason

Mensajes de descubrimiento (UDP): discover, server_info

2.3 Catálogo de mensajes

Mensajes de descubrimiento (UDP)

campo	tipo	valores / significado
discover cliente → UDP broadcast · fase: cualquiera		
v	entero	versión del protocolo que habla el cliente
server_info servidor → UDP unicast · fase: cualquiera		
v	entero	versión del protocolo que habla el servidor
name, tcp_port	texto, entero	nombre del servidor y puerto TCP donde escucha el juego
state, players	texto, entero	fase actual ("lobby"/"playing") y jugadores conectados

Mensajes de cliente a servidor: **join**, **input**, **interact**

2.3 Catálogo de mensajes

Mensajes de cliente a servidor

campo	tipo	valores / significado
join cliente → servidor · fase: lobby		
v	entero	versión del protocolo que habla el cliente
name	texto	nombre del jugador, máximo 20 caracteres, UTF-8
input cliente → servidor · fase: playing		
dir.x	entero	-1 = izquierda · 0 = quieto · 1 = derecha
dir.y	entero	-1 = arriba · 0 = quieto · 1 = abajo
interact cliente → servidor · fase: playing		
sin campos		intenta tomar la bandera libre o robarla al portador

Mensajes de servidor a cliente: **welcome**, **lobby**, **countdown**, **start**, **state**, **game_over**, **error**

2.3 Catálogo de mensajes

Mensajes de servidor a cliente

campo	tipo	valores / significado
welcome servidor → cliente · fase: lobby		
player_id	texto	identificador único que el servidor le asigna al jugador
config.map_size	número	lado del mapa en unidades lógicas
config.circle_radius	número	radio del círculo central
config.player_radius	número	radio del cuerpo del jugador
config.interact_radius	número	distancia máxima para tomar o robar la bandera
config.speed	número	velocidad de movimiento en unidades por segundo
config.tick_rate	entero	envíos de estado por segundo
lobby servidor → cliente · fase: lobby		
players[].id	texto	identificador de cada jugador conectado
players[].name	texto	nombre visible de cada jugador conectado
countdown servidor → cliente · fase: countdown		
seconds	entero	segundos restantes para el inicio (5, 4, 3, 2, 1)
start servidor → cliente · fase: countdown → playing		
sin campos		marca el inicio exacto de la partida
state servidor → cliente · fase: playing		
flag.owner	texto o nulo	id del portador, o null si la bandera está libre
flag.x, flag.y	número	posición actual de la bandera
players[].id	texto	identificador del jugador
players[].x, players[].y	número	posición actual de ese jugador (1 decimal)
game_over servidor → cliente · fase: playing → finished		
winner	texto	id del jugador que ganó la partida
error servidor → cliente · fase: cualquiera		
reason	texto	motivo del rechazo (ej. "game_started", "server_full")

Valores fijos sugeridos para las constantes del juego

2.3 Catálogo de mensajes

Todos los valores fijos acordados hasta ahora

Constantes del juego (viajan en "welcome" → config)

constante	valor sugerido	significado
map_size	1000	el mapa mide 1000 x 1000 unidades lógicas
circle_radius	300	el círculo central mide 300 unidades de radio
player_radius	15	el cuerpo del jugador mide 15 unidades de radio
interact_radius	40	hasta 40 unidades de distancia para tomar o robar la bandera
speed	200	200 unidades por segundo de velocidad de movimiento
tick_rate	20	20 envíos de estado por segundo

Del servidor (no viajan en config, las aplica el servidor directamente)

constante	valor sugerido	significado
countdown_seconds	5	la cuenta regresiva dura 5 segundos antes de iniciar
discovery_port (UDP)	8888	puerto fijo de descubrimiento, igual para toda la clase

Límites del protocolo

límite	valor sugerido	significado
max_players	100	máximo de jugadores por partida (según el enunciado)
name_max_length	20	largo máximo del nombre del jugador, en caracteres
message_max_size	64 KB	tamaño máximo de un mensaje individual

estos números son los que hemos acordado a lo largo de la reunión — usar los mismos evita que dos proyectos "conectados" vean el mapa, el conteo o los límites a distinta escala

Las 9 combinaciones posibles de dir (x, y)

 (-1, -1)	 (0, -1)	 (1, -1)
 (-1, 0)	 (0, 0) quieto	 (1, 0)
 (-1, 1)	 (0, 1)	 (1, 1)

y negativo = arriba, y positivo = abajo, porque el eje Y del mapa crece hacia abajo (sección 3.2)

3. REGLAS DE LA PARTIDA

3.1 Secuencia de la partida

- i. Búsqueda de servidores disponibles (descubrimiento)
- ii. Lobby / espera al comienzo de la partida
- iii. Countdown
- iv. Inicio de la partida
 - iv.i. Ubicación inicial de los jugadores (spawn aleatorio, fuera del círculo)
 - iv.ii. Movimiento libre de los jugadores
- v. Eventos durante la partida (ocurren en cualquier momento y cualquier cantidad de veces)
 - Captura de bandera
 - Robo de bandera
 - Salida del círculo con la bandera (condición de victoria)
- vi. Fin de la partida / mostrar ganador

3.2 Sistema de coordenadas

Ver PDF que mandó Navarro.

3.3 Constantes del juego

Ver PDF que mandó Navarro. o Mirar **ARRIBA**

4. AUTORIDAD Y SINCRONIZACIÓN

4.1 Autoridad y validaciones

Tabla de autoridad y validaciones				
Acción	Qué envía el cliente	Qué valida el servidor	Resultado	
Movimiento	Dirección <code>dir.x</code> y <code>dir.y</code>	Que los valores sean <code>-1</code> , <code>0</code> o <code>1</code> , que la partida esté activa y que el jugador esté conectado	Calcula la nueva posición	
Movimiento diagonal	Dirección en ambos ejes	Normaliza la dirección para evitar mayor velocidad	Mantiene la misma rapidez en todas las direcciones	
Límites del mapa	No envía información adicional	Comprueba que la posición permanezca entre <code>15</code> y <code>985</code> en ambos ejes	Ajusta la posición si intenta salir del mapa	
Captura de bandera	Mensaje <code>interact</code>	Que la bandera esté libre y que la distancia sea menor o igual a <code>40</code>	Asigna la bandera al jugador	
Robo de bandera	Mensaje <code>interact</code>	Que otro jugador posea la bandera y que la distancia entre ambos sea menor o igual a <code>40</code>	Cambia el propietario de la bandera	
Condición de victoria	No envía un mensaje especial	Que el jugador tenga la bandera y haya salido completamente del círculo	Finaliza la partida y declara al ganador	
Mensaje recibido	Mensaje JSON	Que tenga <code>type</code> , los campos requeridos y los tipos de datos correctos	Procesa el mensaje o responde con un error	
Fase de la partida	Acción correspondiente	Que la acción esté permitida en la fase actual	Acepta o rechaza la acción	

4.2 Sincronización del estado

Aspecto	Decisión
Autoridad principal	El servidor
Posiciones oficiales	Las calcula el servidor
Información enviada por el cliente	Intenciones de movimiento e interacción
Captura, robo y victoria	Las valida el servidor
Límites y velocidad	Los controla el servidor
Función del cliente	Enviar acciones y mostrar el estado recibido

5. MANEJO DE FALLAS

5.1 Manejo de errores

Regla general

Cuando el servidor reciba un mensaje incorrecto, deberá:

1. Detectar el problema.
2. No modificar el estado del juego.
3. Responder con un mensaje `error`.
4. Mantener o cerrar la conexión según la gravedad.

Errores comunes			
Código	Cuándo ocurre	¿Se cierra la conexión?	
<code>INVALID_JSON</code>	El texto recibido no es un JSON válido	No, excepto si ocurre repetidamente	
<code>UNKNOWN_TYPE</code>	El campo <code>type</code> contiene un mensaje desconocido	No	
<code>MISSING_FIELD</code>	Falta un campo obligatorio	No	
<code>INVALID_FIELD</code>	Un campo tiene un valor o tipo incorrecto	No	
<code>INVALID_PHASE</code>	La acción no está permitida en la fase actual	No	
<code>VERSION_MISMATCH</code>	Cliente y servidor usan versiones incompatibles	Sí	
<code>LOBBY_FULL</code>	El servidor alcanzó el máximo de jugadores	Sí	
<code>NAME_INVALID</code>	El nombre está vacío, es muy largo o no es válido	No	
<code>MESSAGE_TOO_LARGE</code>	El mensaje supera el tamaño máximo permitido	Sí	
<code>NOT_JOINED</code>	El cliente intenta jugar antes de enviar <code>join</code>	No	

6.2 Decisiones de empate

Caso	Regla autoritativa	Resultado observable
Dos capturan bandera libre	Primer intento válido por arrival_ordinal; tie imposible, player_id fallback.	Uno recibe flag_changed; el otro queda stale.
Varios roban al mismo portador	Solo un cambio por flag_version.	Después del primer robo, deben pulsar otra vez.
Intento duplicado	action_id cacheado 10 s; idempotente.	No cambia dos veces ni incrementa version.
Cruce ganador y robo en mismo tick	Movimiento y victoria se evalúan antes de interacciones.	El portador que cruzó gana; no se procesa robo.
Mensaje viejo	input_seq/sequence menor o igual se descarta.	No rebobina movimiento.
Frame parcial	Buffer espera bytes restantes.	No se considera comando hasta frame completo.
Cliente lento	Eventos se conservan; snapshots pendientes se coalescen al último.	No crece memoria; puede saltar estados visuales.
Robo desde fuera	Nuevo owner debe registrar transición no-fuera -> fuera.	No gana instantáneamente.

6.3 Propiedades que deben permanecer verdaderas

- En cualquier tick existe como máximo un owner_id no cero.
- flag_version nunca disminuye ni se repite después de un cambio.
- winner_id solo se escribe una vez.
- Un request_id/action_id aplicado no vuelve a mutar estado.
- Un cliente no puede ganar enviando coordenadas o match_ended.
- La misma secuencia de comandos ordenados produce el mismo resultado del dominio.

5.2 Desconexiones

Tabla de desconexiones		
Situación	Acción del servidor	Estado de la bandera
Desconexión en lobby	Eliminar jugador y actualizar lobby	Sin cambios
Desconexión en countdown	Eliminar jugador y comprobar mínimo	Sin cambios
Desconexión durante el juego	Eliminar jugador y continuar partida	Sin cambios si no la tenía
Desconexión del portador	Eliminar jugador	Regresa a (500,500)
Se desconectan todos	Reiniciar la partida y volver al lobby	Regresa a (500,500)
Se desconecta el servidor	Los clientes detienen la partida	No existe estado oficial

5.8 Desconexiones

Caso	Decisión v1
TCP close explícito	Eliminar sesión. En PLAYING, jugador queda inmóvil y se retira inmediatamente.
Timeout 8 s	Mismo tratamiento que close; log reason=timeout.
Portador desconecta	Bandera vuelve al centro, mode=ground, owner=0 y version++.
Servidor desconecta	Clientes muestran SERVER_DISCONNECTED; no hay host migration.
Cliente intenta volver	Rechazo durante PLAYING. Reconnect queda fuera de v1.

6. COMPATIBILIDAD

6.1 Versionado del protocolo

6.2 Límites